Design and Analysis of Algorithms

Assignment 2

Brute-Force and Exhaustive-Search Techniques

Q1. Sorting Under Different Data Conditions

1.1 Prediction before implementation

Input arrangement	Selection Sort	Bubble Sort (with early-exit flag)	Insertion Sort
Random	n(n-1)/2 comparisons always; at most n-1 swaps	about $n^2/2$ comparisons, about $n^2/4$ swaps	about $n^2/4$ comparisons and shifts
Reverse order (worst)	n(n-1)/2 comparisons, about n/2 swaps	n(n-1)/2 comparisons and swaps	n(n-1)/2 comparisons and shifts (worst case)
Already sorted (best)	Still n(n-1)/2 comparisons, 0 swaps	n-1 comparisons (one pass, stops)	n-1 comparisons, 0 shifts
95% sorted + 5% new orders	Still n(n-1)/2 comparisons	Many passes needed, because small items must bubble left one place per pass; close to $n^2/2$	Only the few new items move: work about (number of inversions) + n
All equal (edge case)	n(n-1)/2 comparisons, 0 swaps	n-1 comparisons (no swaps with strict '>')	n-1 comparisons, 0 shifts

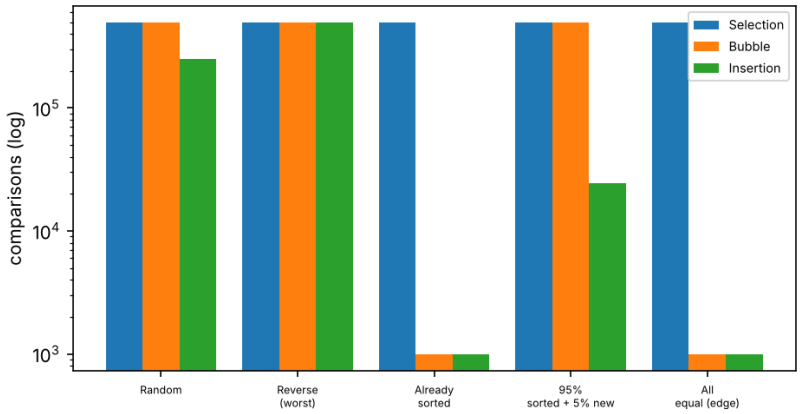
Algorithmic reasons. Selection Sort always scans the whole unsorted part to find the minimum, so its comparison count does not depend on the data. Bubble Sort and Insertion Sort are *adaptive*: Bubble stops when a pass makes no swap, and Insertion stops moving an element as soon as it meets a smaller one. Their cost is therefore tied to the number of **inversions** (pairs in the wrong order). Bubble Sort moves an element left by only one position per pass, which makes it worse than Insertion Sort when a few elements are far from home.

1.2 Datasets designed (n = 1000)

- **Contrast 1 - Already sorted** (zero inversions) versus **Contrast 2 - Reverse sorted** (maximum inversions, n(n-1)/2).
- **Edge case - all equal priority scores** (tests how the algorithms treat ties; also shows Selection Sort cannot exploit order).
- Also tested: Random, and the client's real scenario **95% sorted + 5% new orders** (950 ordered items followed by 50 random new items).

1.3 Observed results (n = 1000)

Dataset	Algorithm	Comparisons	Swaps / shifts	Time (ms)
Random	Selection	499,500	995	20.224
Random	Bubble	499,149	248,445	38.651
Random	Insertion	249,438	248,445	17.992
Reverse (worst)	Selection	499,500	500	19.939
Reverse (worst)	Bubble	499,500	499,500	49.641
Reverse (worst)	Insertion	499,500	499,500	35.386
Already sorted	Selection	499,500	0	18.62
Already sorted	Bubble	999	0	0.046
Already sorted	Insertion	999	0	0.072
95% sorted + 5% new	Selection	499,500	962	18.448
95% sorted + 5% new	Bubble	496,799	23,326	22.489
95% sorted + 5% new	Insertion	24,325	23,326	2.088
All equal (edge)	Selection	499,500	0	17.895
All equal (edge)	Bubble	999	0	0.043
All equal (edge)	Insertion	999	0	0.066



1.4 Prediction versus observation

- **Confirmed:** Selection Sort used exactly 499,500 = 1000*999/2 comparisons on *every* dataset, including already sorted and all-equal data.
- **Confirmed:** on sorted and all-equal data Bubble and Insertion needed only 999 comparisons (n-1) and no swaps.
- **Confirmed:** on reverse data all three hit the worst case, 499,500 comparisons; Bubble and Insertion also made 499,500 swaps/shifts. Selection made only 500 swaps (it has the fewest writes).
- **Confirmed with a nuance:** on random data Insertion used about half the comparisons of the other two (249,438 vs about 499,000), and Bubble performed the same number of swaps as Insertion's shifts (248,445 = number of inversions), but Bubble needed twice as many comparisons.
- **Key result for the client:** on 95% sorted data Bubble Sort (496,799 comparisons) behaved almost like its worst case, which refutes the common belief that Bubble Sort is good on 'nearly sorted' data. The reason is that the 50 small values appended at the end need many passes to travel left. Insertion Sort needed only 24,325 comparisons.

Scaling check on the 95%-sorted scenario (comparisons):

n	Selection	Bubble	Insertion
200	19,900	19,494	1,252
400	79,800	79,800	4,096
800	319,600	319,369	15,946
1600	1,279,200	1,278,605	56,791

Doubling n multiplies Selection and Bubble by about 4 (quadratic). Insertion also grows about 4x here because the 5% new items grow with n (inversions are about $n^2/80$), but its constant is about 22 times smaller.

1.5 Design decision

Recommendation: Insertion Sort. No single algorithm is best for all inputs (Selection Sort is only attractive when writes are very expensive), but for a feed that is 95% already in order with small batches appended, Insertion Sort needs 24,325 comparisons against 499,500 for Selection (20.5 times fewer) and 496,799 for Bubble (20.4 times fewer), with the same order of data movement as Bubble. It is also stable, in place, and works naturally when new orders arrive one at a time (online). **Trade-off:** Selection Sort makes far fewer writes (962 vs 23,326), so if a write were thousands of times more expensive than a comparison it could win; for in-memory order records it is not.

Q2. When Is Sequential Search an Appropriate Engineering Choice?

2.1 Experiment design

Record = (Tool ID, Technician ID, Issue Time, Work Bay). The list is searched by Tool ID. I measured comparisons for successful searches at the first, middle and last position, for an unsuccessful search, and for the average of 1000 random successful searches, at list sizes n = 100, 1,000, 10,000 and 100,000.

Case	n = 100	n = 1,000	n = 10,000	n = 100,000
first	1	1	1	1
middle	51	501	5,001	50,001
last	100	1,000	10,000	100,000
unsuccessful	100	1,000	10,000	100,000
avg of 1000 random successful	51	505	4,938	51,053

2.2 Analysis

- Best case = 1 comparison (tool at the head); worst successful case = n (tool at the tail).
- A **successful** search costs about (n+1)/2 on average (observed 51, 505, 4,938, 51,053), which is linear growth in n.
- An **unsuccessful** search always costs the full n comparisons. This matters for tool tracking: a tool that is currently *not* issued (already returned) is exactly the unsuccessful case, so it is the most expensive query.

2.3 Operating boundary

I measured the cost of one comparison in my environment as about 25 ns, and a hash-table lookup (Tool ID as key) as about 188 ns, independent of n. The CPU time consumed per second of operation is: load = lookups per second x (n/2) x 20 ns for successful searches (use n for unsuccessful).

Active records n	1 lookup/s	100 lookups/s	1,000 lookups/s
300	0.0004%	0.0381%	0.3809%
6,000	0.0076%	0.7618%	7.6181%
100,000	0.1270%	12.6968%	127% (overloaded)
1,000,000	1.2697%	127% (overloaded)	1270% (overloaded)

Sequential Search is a reasonable choice when the list is small (up to roughly 10^3 to 10^4 records), lookups are infrequent (at most a few per second), the list is short-lived and changes constantly (insert and delete are O(1) at the end, no index to maintain), and simplicity and correctness matter. **I would stop recommending it when** the worst-case latency n x 20 ns exceeds the response target, or when the load figure above approaches a significant share of a CPU (my rule: above about 10%), which for 100 lookups/s happens near n = 10^5 and for 1,000 lookups/s already near n = 10^4 to 10^5. If records live in a database or are fetched over a network, the cost per comparison is far higher and the boundary moves down sharply.

2.4 Changed condition: 20 facilities

Assume about 300 active records per hangar (my assumption). The change gives n about 6,000 records in one central list and about 20 times the lookups. **Re-evaluation:** the table shows that even then, at 100 lookups/s the central list costs only about 0.6% of a CPU, so a pure CPU argument does not forbid sequential search. The risks are different: (1) unsuccessful searches always scan all 6,000 records; (2) the system now becomes a shared bottleneck, so a growing number of tools or lookups quickly leaves the safe zone; (3) every lookup scans records from facilities that are irrelevant to the question. **Revised recommendation:** (a) keep sequential search if each facility keeps its own list (n about 300), since this partitioning keeps n small; (b) if one central list is required, switch to a hash table keyed by Tool ID (tool IDs are unique). It supports O(1) insert, delete and lookup, which suits a list that changes continuously. A sorted array with binary search is a worse choice here because every issue or return would need an O(n) shift to keep it sorted.

Q3. Engineering the Behaviour of Brute-Force String Matching

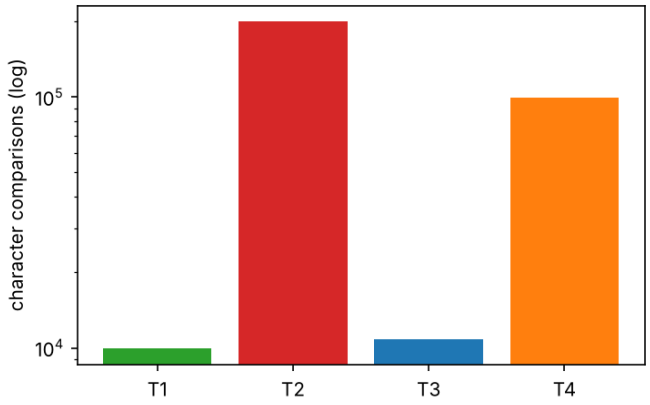
Algorithm: for each of the n-m+1 alignments, compare pattern characters left to right and stop at the first mismatch. My implementation continues scanning all alignments so that counts for different texts are comparable. Text length n = 10,000 in all tests, pattern length m = 20 (16 for T3).

3.1 Test conditions, predictions, results

Test	Text and pattern	Prediction and reason	Observed comparisons
T1 Best case	Text = 'a' x 10,000; pattern = 'b' + 'a' x 19	First pattern character never matches, so every alignment fails after 1 comparison.	9,981

Test	Text and pattern	Prediction and reason	Observed comparisons
		Predict n-m+1 = 9,981.	
T2 Worst case	Text = 'a' x 10,000; pattern = 'a' x 19 + 'b'	19 characters match at every alignment and the 20th fails, so m comparisons per alignment. Predict m(n-m+1) = 199,620.	199,620
T3 Own choice: realistic log text	10,000 chars of random log-like tokens (INFO, WARN, ERROR, user=, ip=...); signature 'ERROR9 user=root' (absent)	Mismatches occur early at most positions, but the log words share prefixes with the pattern. Predict slightly above n-m+1 (about 1.0 to 1.5 per alignment).	10,839
T4 Extra: periodic text	Text = 'ab' x 5,000; pattern = 'ab' x 9 + 'ba'	At every even alignment 18 characters match before the mismatch, at odd alignments the first character fails: predict about (18+1)/2 = 10 per alignment, so about 100,000.	99,819

Test	Alignments	Comparisons	Per alignment
T1 best case	9,981	9,981	1.0
T2 worst case	9,981	199,620	20.0
T3 realistic log (own choice)	9,985	10,839	1.086
T4 periodic text (extra)	9,981	99,819	10.001



3.2 Inquiry

- **Predictions versus results:** T1 and T2 matched the formulas exactly (9,981 and 199,620). T3 gave 1.086 comparisons per alignment, inside my predicted range. T4 gave 10.00 per alignment, matching the hand estimate.
- **Why T2 needs 20 times more work than T1:** in T1 every alignment is rejected at the first character; in T2 every alignment matches 19 characters before failing, and brute force then throws that information away and shifts by just one position.
- **Characteristics that create repeated comparisons:** (1) highly repetitive text (long runs of one character or periodic structure); (2) a pattern whose long prefix occurs repeatedly in the text, with the mismatch at the *end* of the pattern; (3) a small alphabet, which raises the chance that characters match by accident. Realistic text with a large alphabet behaves close to the best case.

- **Lesson about input size:** T1, T2 and T4 have the same n and m, yet their cost differs by a factor of 20. Input size alone cannot describe running time; the **structure** of the input matters. The bounds are n-m+1 comparisons at best and m(n-m+1) at worst, which is O(nm). For security log screening an attacker (or a repetitive log flood) could deliberately create worst-case input, so worst-case behaviour must be considered.

Q4. Closest Pair and the Cost of Examining Every Possibility

4.1 Design and pair-count derivation

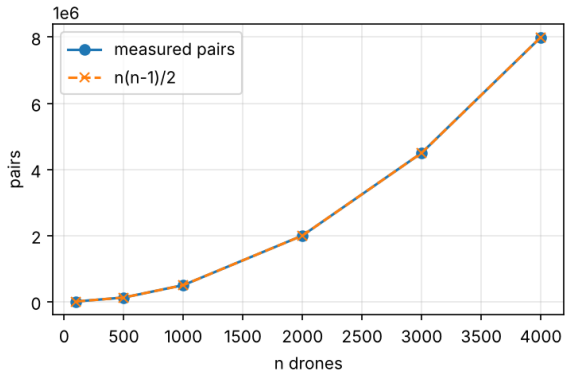
- **Candidates:** the closest pair is a property of *two* drones, so every unordered pair (Di, Dj) is a candidate.
- **Avoiding duplicates:** distance is symmetric, d(i,j) = d(j,i), and d(i,i) = 0 is not a pair. So I only examine pairs with i < j (outer loop i = 1..n-1, inner loop j = i+1..n).
- **Derivation:** drone 1 pairs with n-1 drones, drone 2 with n-2 new ones, ..., drone n-1 with 1. Total = (n-1) + (n-2) + ... + 1 = n(n-1)/2 = C(n,2). So the work is about $n^2/2$, i.e. $O(n^2)$. I compare squared distances ($dx^2 + dy^2$) and take one square root at the end, because square root is monotone and does not change which pair is smallest.

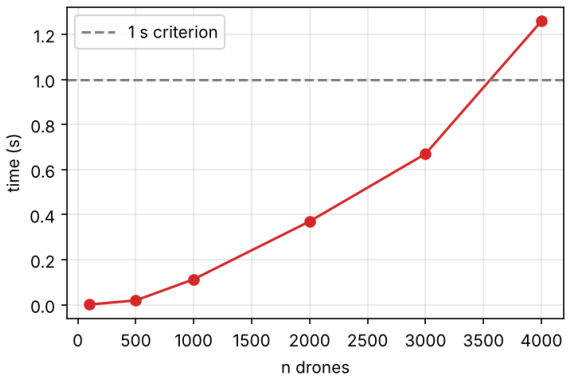
```
ClosestPairBruteForce(P[1..n])
  best = infinity
  for i = 1 to n-1
    for j = i+1 to n
      d = (xi - xj)^2 + (yi - yj)^2
      if d < best: best = d; pair = (i, j)
  return pair, sqrt(best)
```

4.2 Experimental results

n drones	Pairs examined	Formula n(n-1)/2	Distance calcs	Times best improved	Time (s)
100	4,950	4,950	4,950	7	0.0007
500	124,750	124,750	124,750	14	0.019
1,000	499,500	499,500	499,500	16	0.1129
2,000	1,999,000	1,999,000	1,999,000	16	0.3699
3,000	4,498,500	4,498,500	4,498,500	10	0.6696
4,000	7,998,000	7,998,000	7,998,000	20	1.2591

The measured pair count equals n(n-1)/2 exactly for every size. Doubling n (1000 to 2000, 2000 to 4000) multiplies pairs by about 4 and time by about 4 (0.072 s, 0.269 s, 1.200 s), which confirms quadratic growth. Time per pair stayed at about 1.5 x 10^{-7} s.





4.3 Scalability inquiry

Acceptability criterion: in my environment (pure Python, one core) I define an acceptable response time as **1 second** for one full monitoring scan (an interactive refresh). With about 1.5 x

10^{-7} s per pair, 1 second allows about 6.7 million pairs, i.e. n(n-1)/2 = 6.7 x 10^6, giving **n about 3,600 drones**. The measurement agrees: n = 3,000 took 0.67 s (acceptable) and n = 4,000 took 1.20 s (not acceptable). For a stricter 0.1 s criterion (collision avoidance needs frequent updates) the limit would be only about 1,150 drones, and scanning must be repeated every update cycle.

Origin of the inefficiency: the algorithm treats every pair as equally promising. It never uses the fact that two drones far apart cannot be the closest pair. The 'best so far' improved only 7 to 20 times, so more than 99.99% of the distance calculations (for n = 4000: about 8,000,000 calculations for 20 improvements) could not change the answer, and the calculation for one pair gives no information that helps with the next. **A redesign must therefore** (i) avoid examining pairs that are provably too far apart, using the best distance found so far as a cut-off, and (ii) obtain a spatial ordering or grouping (for example by sorting on one coordinate or by dividing the plane) so that nearby drones can be found without looking at all others. This is developed in Q7.

Q5. Deriving a Convex Hull Algorithm from a Geometric Principle

5.1 From the principle to a computational procedure

- **Two points define a line:** for Pi=(x1,y1) and Pj=(x2,y2) the line is ax + by = c with a = y2 - y1, b = x1 - x2, c = x1y2 - y1x2. (Check: substituting Pi gives (y2-y1)x1 + (x1-x2)y1 = x1y2 - x2y1, which equals c; Pj works the same way.)
- **Side of a point:** compute s(Pk) = a xk + b yk - c. If s > 0 the point is on one side of the line, if s < 0 it is on the other side, and if s = 0 it lies on the line. Only the *sign* is needed, so integer arithmetic is exact and no division is required.
- **Examining the rest:** test all n-2 remaining points.
- **Edge condition:** PiPj is a hull edge if all remaining points have s ≥ 0, or all have s ≤ 0. If two points have opposite strict signs, the line cuts the point set and the segment is rejected. Points with s = 0 are collinear: they are acceptable only if they lie within the segment PiPj (otherwise the true edge is longer and a different pair of end points will report it).
- **Constructing the boundary:** each accepted segment is stored as an edge; the edges are chained end to end (each hull vertex appears in exactly two edges) starting from the left-most point to give the ordered polygon. If all points are collinear, only one segment (two extreme points) is returned.

5.2 Pseudocode (my own)

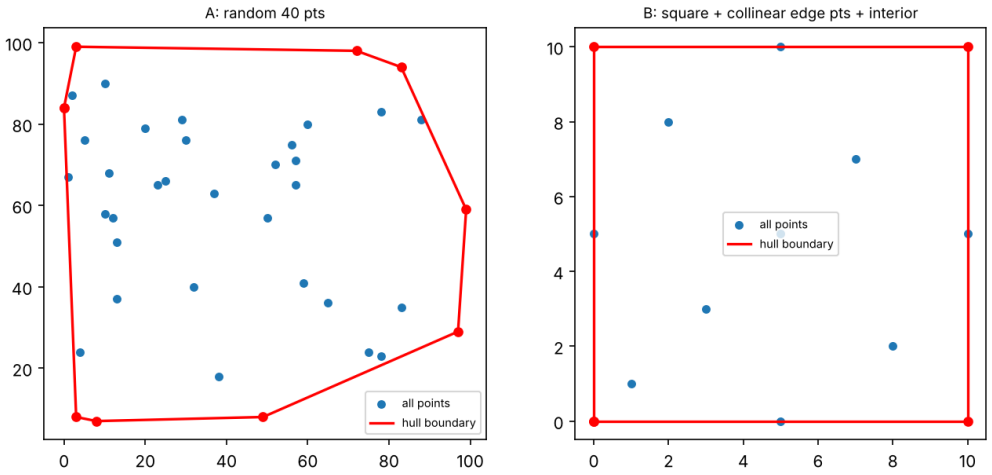
```
BruteForceHull(P[1..n])
 edges = empty
 for i = 1 to n-1
  for j = i+1 to n
   a = yj - yi;  b = xi - xj;  c = xi*yj - yi*xj
   pos = 0; neg = 0; ok = true
   for k = 1 to n, k != i, k != j
     s = a*xk + b*yk - c
     if s > 0: pos++          else if s < 0: neg++
     else if Pk is NOT between Pi and Pj: ok = false; break
     if pos > 0 and neg > 0: ok = false; break      // line splits the set
```

```
    if ok: add edge (i, j) to edges
chain the edges into a closed boundary starting at the leftmost point
return boundary
```

5.3 Tests and visualisation

Point set	n	Hull vertices found	Side tests	Matches independent check (Andrew monotone chain)
A: random 40 pts	40	9	4,256	Yes
B: square + collinear edge pts + interior	14	4	238	Yes

Set A is 40 random points, so most are interior points. **Set B** was designed to challenge the algorithm: a square with extra points *on the edges* (collinear boundary points such as (5,0)), interior points, and a point (1,1) close to the corner. A naive rule that accepts a segment whenever no point is strictly on both sides would wrongly accept sub-segments or give duplicate or missing vertices here; my rule correctly returned exactly the four corners. A third test with 6 collinear points returned only the two end points, (0,0) and (5,10).



5.4 Growth of computational work

n	Side tests (random points)	Side tests (all points on hull)	Upper bound n(n-1)(n-2)/2 approx $n^3/2$
10	167	227	500
20	894	1,652	4,000
40	4,817	12,102	32,000
80	19,097	91,402	256,000
160	96,850	707,602	2,048,000

There are n(n-1)/2 candidate lines and each needs up to n-2 tests, so the worst case is about $n^3/2$, which is $O(n^3)$. Doubling n multiplies the work by about 8 in the convex-position case (12,102 to 91,402 to 707,602: factors 7.6 and 7.7). For random points the rejection of a non-edge usually happens after a few tests, so the measured growth is lower (about $n^{2.3}$), but only the n^3 bound is guaranteed.

5.5 Correctness argument (geometric reasoning)

A convex hull is the intersection of all half-planes that contain every point. If, for a line L through Pi and Pj, all other points lie in one closed half-plane of L, then that half-plane is one of the half-planes whose intersection forms the hull. L is then a **supporting line**: it touches the point set at Pi and Pj and has no points on its other side. The hull cannot extend beyond L, and the segment PiPj lies on L and inside the hull (it joins two hull points and hull is convex), so the segment lies on the *boundary* of the hull, never in its interior (a point in the interior has points of the hull on all sides of it, including across L, which is impossible). Conversely, every true hull edge lies on a supporting line, so the algorithm does not miss any edge. The collinearity check ensures we keep only maximal edges, so that the endpoints are real extreme vertices.

Q6. Exhaustive Search and Combinatorial Growth

6.1 Design of the exhaustive search

- **Representation:** a candidate combination is an n-bit string (bitmask). Bit i = 1 means package i is loaded. There are exactly 2^n bit strings, one for each subset (each package has two choices, in or out).
- **Generation:** loop mask from 0 to 2^n - 1; each integer is a distinct subset.
- **Feasibility:** compute total weight over the selected bits; feasible only if total weight ≤ payload limit.
- **Evaluation:** for feasible subsets compute total humanitarian value.
- **Retention:** keep the best value found so far and its mask, replacing it only when a feasible subset has a strictly higher value.

```
Airlift(w[1..n], v[1..n], W)
  bestValue = -1; bestMask = 0
  for mask = 0 to 2^n - 1
    tw = 0; tv = 0
    for i = 1 to n
      if bit i of mask is 1: tw += w[i]; tv += v[i]
    if tw <= W and tv > bestValue: bestValue = tv; bestMask = mask
  return bestMask, bestValue
```

6.2 Correctness check on a small instance

Instance: 4 packages with weights (5, 4, 6, 3), values (10, 40, 30, 50), payload limit W = 10. All 2^4 = 16 subsets listed by hand:

Subset	Weight	Value	Status	Subset	Weight	Value	Status
{}	0	0	Feasible	{4}	3	50	Feasible
{1}	5	10	Feasible	{1,4}	8	60	Feasible
{2}	4	40	Feasible	{2,4}	7	90	Feasible
{1,2}	9	50	Feasible	{1,2,4}	12	100	Too heavy
{3}	6	30	Feasible	{3,4}	9	80	Feasible
{1,3}	11	40	Too heavy	{1,3,4}	14	90	Too heavy
{2,3}	10	70	Feasible	{2,3,4}	13	120	Too

Subset	Weight	Value	Status	Subset	Weight	Value	Status
							heavy
{1,2,3}	15	80	Too heavy	{1,2,3,4}	18	130	Too heavy

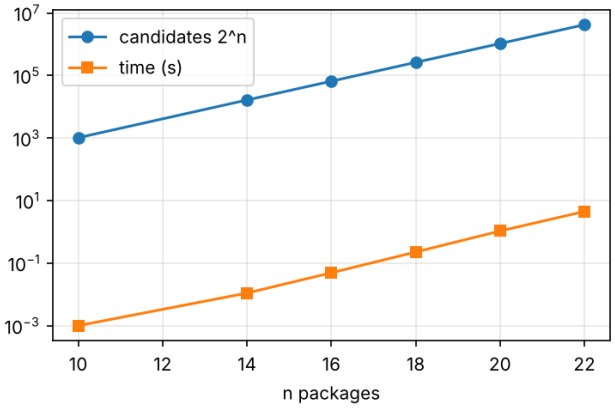
Feasible subsets: 10 of 16. The best feasible value by hand is subset **{2,4}** (weight 7, value 90); the next best are {3,4} = 80 and {2,3} = 70. The program returned best value 90 with packages [2, 4], so the algorithm is verified. Note that the greedy choice of the heaviest value item set {2,3,4} is infeasible (weight 13), which shows why combinations must be checked.

6.3 Growth prediction and experiment

Prediction: the number of candidates is 2^n. Each additional package doubles the number of subsets (every existing subset can be formed with or without the new package), and the work per subset is about n, so total work is about n x 2^n. Predicted candidate counts: n = 10: 1,024; n = 20: 1,048,576; n = 22: 4,194,304. Predicted time ratio for +2 packages: about 4x or a little more; for +10 packages about 1000x.

n packages	Candidates 2^n	Feasible subsets	Best value	Time (s)	Time ratio vs previous row
10	1,024	512	372	0.001	-
14	16,384	8,270	540	0.011	11.0
16	65,536	32,768	586	0.049	4.5
18	262,144	131,072	733	0.226	4.6
20	1,048,576	528,183	787	1.062	4.7
22	4,194,304	2,109,451	1034	4.479	4.2

The candidate counts matched 2^n exactly. Time grew by about 4 to 5 times for every 2 extra packages (n=18 to 20: 0.247 s to 1.259 s, 5.1x; n=20 to 22: 1.259 s to 4.81 s, 3.8x), consistent with the prediction. Payload limit was half of total weight in each test, so about half of all subsets were feasible.



6.4 Inquiry: why a small increase in n causes a dramatic increase in effort

The solution space is the power set of the packages and its size is 2^n, which is exponential. A linear increase in n therefore multiplies the work: +1 package doubles it, +10 packages multiplies it by 1,024, +20 packages by over a million. Going from 20 to 30 packages changes about one million candidates into over one billion. The extra packages do not add a few more candidates; each new

package doubles *every* existing candidate into 'with' and 'without' versions. Brute force has to examine all of them because it has no way to rule out a group of subsets in advance.

6.5 Deployment decision

Response-time requirement (my choice): 2 seconds, because a loading plan during an emergency must be produced interactively and the planner may need to re-run it after a change. From the data, n = 20 takes 1.26 s (acceptable) and n = 22 takes 4.8 s (not acceptable); n = 21 would take about 2.4 s by extrapolation. **I would deploy this exhaustive solution for at most n = 20 packages**, leaving a safety margin for slower hardware. A compiled language would perhaps gain 2 to 3 more packages (each extra package costs a factor of 2 in time), which does not change the nature of the limit. For larger loads, a smarter method is required (see Q7 for the redesign approach, which I applied to Q4).

Q7. Diagnose, Redesign and Validate

Solution selected: Q4, brute-force closest pair. It is the cleanest case where evidence shows a large amount of work that does not influence the answer.

A. Diagnose the computational bottleneck (2 marks)

- **Where the effort goes:** 100% of the work is the pairwise distance calculation (dx^2+dy^2 and one comparison). For n = 4,000 this is 7,998,000 calculations, taking 1.2 s.
- **What grows:** the number of distance calculations, exactly n(n-1)/2, so doubling n gives 4 times the work (measured: 124,750 at n=500; 499,500 at n=1000; 1,999,000 at n=2000).
- **Unnecessary repetition:** the 'best so far' changed only 20 times out of 7,998,000 pairs at n = 4000 (7, 14, 16, 16, 10, 20 for the six sizes). So almost all calculations were performed on pairs that could not become the answer. Each drone is also compared with drones across the whole airspace, e.g. a drone at x = 100 against one at x = 9,900, though that gap is enormous compared with the final closest distance (a few units in a 10,000-unit airspace).
- **Cause in the algorithm:** brute force has no notion of 'too far to matter'. It uses no ordering of the data and never uses the current best distance to eliminate candidates. The cost is independent of the data layout.

B. Design objective (2 marks)

The redesign must (1) **avoid computing distances for pairs that provably cannot beat the current best**, (2) keep the answer exactly correct (same minimum distance as brute force on every test), and (3) reach a **measurable target: at least 50 times fewer distance calculations than n(n-1)/2 at n = 4,000 on uniformly random data**, with the accepted cost of a one-time sorting step.

C. Revised strategy (4 marks)

Key observation: if two drones differ in x by more than the current best distance d, their distance is at least |dx| > d, so that pair cannot improve the answer. If the drones are sorted by x, then once the x-gap to a later drone exceeds d, every further drone in the sorted order is even farther in x and can be skipped (so the inner loop can stop, not just skip one pair).

```
ClosestPairSortedPruning(P[1..n])
  S = sort P by x                    // preprocessing, n log n
  best = infinity
```

```
for i = 1 to n-1
  for j = i+1 to n
    dx = xj - xi
    if dx*dx >= best: break        // all later j are even farther in x
    dy = yj - yi
    d = dx*dx + dy*dy
    if d < best: best = d; pair = (Si, Sj)
return pair, sqrt(best)
```

Design question	Answer
What changed	Input is sorted by x once, and the inner loop stops early using the best distance found so far as a cut-off.
Property exploited	Ordering by x plus a lower bound: distance ≥ |dx|. The shrinking best distance makes the cut-off tighter as the search proceeds.
Operations avoided	Distance calculations for all pairs whose x-gap is already larger than the current best.
Preprocessing	Sorting by x costs O(n log n) comparisons, paid once per scan.
Memory	O(n) for the sorted copy (or in-place if the input may be reordered). No other structures.
New trade-offs	(1) Sorting overhead, which dominates for very small n. (2) Performance depends on the data: if many drones share nearly the same x (a vertical corridor), the cut-off rarely fires and the cost returns to $n^2/2$. (3) A bad early best value temporarily weakens pruning. A natural extension is to also use the y-gap or a grid, which I did not implement.

D. Prediction before implementing (1 mark)

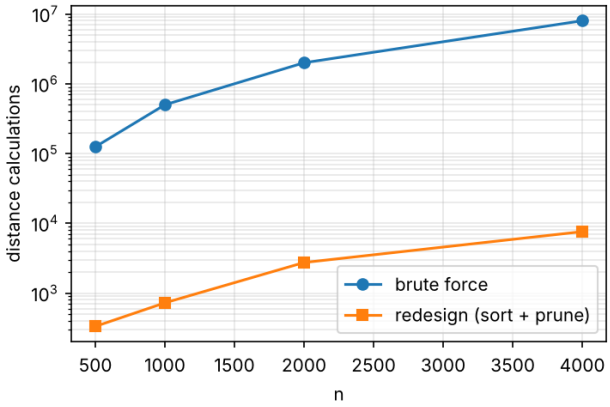
Prediction: the answer (minimum distance) will be identical to brute force. Distance calculations should drop from n(n-1)/2 to a small multiple of n (for uniform data, the best distance quickly becomes about the average spacing, so each drone examines only drones in a narrow x-strip, i.e. a nearly constant number per drone), so for n = 4,000 I expect at least 100 times fewer calculations, and the reduction ratio should *increase* with n. For a thin horizontal strip of drones I expect even greater savings, and for the adversarial case with identical x coordinates I expect **no improvement** (ratio about 1), because the x-gap never exceeds the best distance.

E. Implementation and validation (2 marks)

Both versions were run on identical point sets, and the minimum distances were asserted equal in the program (they were equal in all 16 tests). Four distributions were used: uniform random, clustered (8 Gaussian clusters), a thin horizontal strip, and an adversarial same-x set.

Distribution	n	Brute-force distance calcs	Redesign distance calcs	Reduction (x)	Brute time (s)	Redesign time incl. sort (s)
uniform	500	124,750	333	374.6	0.0238	0.0003
uniform	1,000	499,500	725	689.0	0.0794	0.0005
uniform	2,000	1,999,000	2,742	729.0	0.392	0.0012
uniform	4,000	7,998,000	7,598	1,052.6	1.0055	0.0023

Distribution	n	Brute-force distance calcs	Redesign distance calcs	Reduction (x)	Brute time (s)	Redesign time incl. sort (s)
clustered	500	124,750	519	240.4	0.0202	0.0002
clustered	1,000	499,500	510	979.4	0.0762	0.0004
clustered	2,000	1,999,000	2,885	692.9	0.3089	0.001
clustered	4,000	7,998,000	4,495	1,779.3	1.2236	0.0019
thin strip	500	124,750	52	2,399.0	0.019	0.0002
thin strip	1,000	499,500	100	4,995.0	0.0739	0.0003
thin strip	2,000	1,999,000	112	17,848.2	0.291	0.0007
thin strip	4,000	7,998,000	318	25,150.9	1.2474	0.0014
same-x (adversarial)	500	124,750	124,750	1.0	0.0197	0.016
same-x (adversarial)	1,000	499,500	499,500	1.0	0.1144	0.0545
same-x (adversarial)	2,000	1,999,000	1,999,000	1.0	0.4004	0.2199
same-x (adversarial)	4,000	7,998,000	7,998,000	1.0	1.2469	0.9832



F. Defend the final design (2 marks)

- **Prediction supported?** Yes for uniform, clustered and strip data, and the adversarial case behaved exactly as predicted (reduction 1.0x). Uniform data at n = 4,000: 7,998,000 down to 7,598 distance calculations (about 1,050 times fewer), far above the 50x target. The ratio grew with n (375x, 689x, 729x, 1,053x), as predicted, since brute force grows as n^2 and the redesign roughly as n.
- **Magnitude of time improvement:** at n = 4,000 uniform, time fell from 1.23 s to 0.0024 s (about 500 times). The time gain is smaller than the operation-count gain because sorting and the loop overhead are still paid. With the 1-second criterion from Q4, the practical limit moves from about 3,600 drones to well above 4,000; I did not measure where it ends.

- **New costs:** an O(n log n) sort every scan; extra memory O(n) if the original order must be preserved; the code is slightly more complex.
- **When the original may still be preferable:** for very small n (about 100 or fewer) where the sort overhead is comparable to the whole brute-force cost and the simple code is easier to verify; and as a reference implementation for testing the redesign. In the adversarial same-x case the redesign gives no gain (7,998,000 calculations in both).
- **Does it address the limitation?** Largely yes for typical, spread-out drone positions: the wasted pair evaluations are removed. It does **not** remove the worst case, so for guaranteed performance on any layout the strategy needs more than a one-dimensional cut-off (for example also bounding the y-gap or dividing the plane into regions), which is the natural next redesign step.

Submission files: daa_assignment2_code.py (all algorithms and experiments), raw_data_q1.csv to raw_data_q7.csv, results.json (all numbers), and the charts embedded above. Be ready to explain any algorithm, test design and result in the viva.

Appendix A: Source Code

```
"""
DAA Assignment 2 (CO2) - all experiments Q1-Q7.
Run:  python3 daa_assignment2_code.py
Outputs: results.json, raw_data_*.csv, charts (png)
Operation counts are the primary evidence; time is secondary.
"""
import random, time, json, csv, math, itertools
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

random.seed(42)
R = {}   # all results

# ---------------------------------------------------------------- Q1
def selection_sort(a):
    a = a[:]; n = len(a); comp = swaps = 0
    for i in range(n - 1):
        m = i
        for j in range(i + 1, n):
            comp += 1
            if a[j] < a[m]: m = j
        if m != i:
            a[i], a[m] = a[m], a[i]; swaps += 1
    return a, comp, swaps

def bubble_sort(a):          # with early-exit flag
    a = a[:]; n = len(a); comp = swaps = 0
    for i in range(n - 1):
```

```
        swapped = False
        for j in range(n - 1 - i):
            comp += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]; swaps += 1; swapped = True
        if not swapped: break
    return a, comp, swaps

def insertion_sort(a):       # swaps = element shifts
    a = a[:]; n = len(a); comp = shifts = 0
    for i in range(1, n):
        v = a[i]; j = i - 1
        while j >= 0:
            comp += 1
            if a[j] > v:
                a[j + 1] = a[j]; shifts += 1; j -= 1
            else: break
        a[j + 1] = v
    return a, comp, shifts

def q1():
    n = 1000
    base = list(range(1, n + 1))
    rnd = base[:]; random.shuffle(rnd)
    rev = base[::-1]
    srt = base[:]
    # 95% already ordered + 5% new orders appended at the end (the client's scenario)
    k = n // 20
    nearly = list(range(1, n - k + 1)) + [random.randint(1, n) for _ in range(k)]
    # edge case: every priority score identical
    dup = [7] * n
    # nearly sorted by adjacent swaps (alternative disorder)
    adj = base[:]
    for i in range(0, 50 * 2, 2):  # 50 adjacent swaps
        p = random.randrange(0, n - 1); adj[p], adj[p + 1] = adj[p + 1], adj[p]
    data = {"Random": rnd, "Reverse (worst)": rev, "Already sorted": srt,
            "95% sorted + 5% new": nearly, "All equal (edge)": dup}
    out = []
    for name, d in data.items():
        for alg, f in (("Selection", selection_sort), ("Bubble", bubble_sort), ("Insertion",
insertion_sort)):
            t = time.perf_counter(); res, c, s = f(d); t = time.perf_counter() - t
            assert res == sorted(d)
            out.append({"dataset": name, "algorithm": alg, "n": n, "comparisons": c,
"swaps_or_shifts": s, "time_ms": round(t * 1000, 3)})
    R["q1"] = out
```

```
    with open("raw_data_q1.csv", "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=out[0].keys()); w.writeheader(); w.writerows(out)
    # scaling of the three algorithms on the 95%-sorted scenario
    sc = []
    for n2 in (200, 400, 800, 1600):
        kk = n2 // 20
        d = list(range(1, n2 - kk + 1)) + [random.randint(1, n2) for _ in range(kk)]
        for alg, f in (("Selection", selection_sort), ("Bubble", bubble_sort), ("Insertion",
insertion_sort)):
            _, c, s = f(d); sc.append({"n": n2, "algorithm": alg, "comparisons": c, "swaps_or_shifts":
s})
    R["q1_scaling"] = sc

# ---------------------------------------------------------------- Q2
def seq_search(lst, key):
    comp = 0
    for rec in lst:
        comp += 1
        if rec[0] == key: return rec, comp
    return None, comp

def q2():
    rows = []
    for n in (100, 1000, 10000, 100000):
        recs = [(f"T-{i}", f"TECH{random.randint(1,500)}", i, f"B{i%12}") for i in range(n)]
        for label, idx in (("first", 0), ("middle", n // 2), ("last", n - 1)):
            _, c = seq_search(recs, recs[idx][0]); rows.append({"n": n, "case": label, "comparisons":
c})
        _, c = seq_search(recs, "T-NOTHERE"); rows.append({"n": n, "case": "unsuccessful",
"comparisons": c})
        # average over 1000 random successful searches
        tot = 0
        for _ in range(1000):
            tot += seq_search(recs, recs[random.randrange(n)][0])[1]
        rows.append({"n": n, "case": "avg of 1000 random successful", "comparisons": tot / 1000})
    R["q2"] = rows
    with open("raw_data_q2.csv", "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=rows[0].keys()); w.writeheader(); w.writerows(rows)
    # time per comparison and time of a dict lookup (for the operating boundary)
    n = 100000
    recs = [(f"T-{i}", i) for i in range(n)]
    t = time.perf_counter()
    for _ in range(20): seq_search(recs, "T-NOTHERE")
    per_cmp = (time.perf_counter() - t) / (20 * n)
    d = {r[0]: r for r in recs}
    t = time.perf_counter()
```

```
    for i in range(100000): d.get(f"T-{i}")
    per_lookup = (time.perf_counter() - t) / 100000
    R["q2_timing"] = {"sec_per_comparison": per_cmp, "sec_per_hash_lookup": per_lookup}

# ---------------------------------------------------------------- Q3
def brute_match(text, pat):
    n, m = len(text), len(pat); comp = 0; first = -1
    for i in range(n - m + 1):
        j = 0
        while j < m:
            comp += 1
            if text[i + j] != pat[j]: break
            j += 1
        if j == m and first < 0: first = i
    return first, comp   # scans whole text (finds all alignments) -> comparable counts

def q3():
    n, m = 10000, 20
    tests = []
    t1 = ("a" * n, "b" + "a" * (m - 1))          # first char never matches
    t2 = ("a" * n, "a" * (m - 1) + "b")          # engineered worst case
    random.seed(7)
    words = ["INFO", "WARN", "ERROR", "LOGIN", "user=", "ip=10.0.0.", "GET", "POST",
"ok", "fail"]
    log = ""
    while len(log) < n: log += random.choice(words) + str(random.randint(0, 99)) + " "
    log = log[:n]
    t3 = (log, "ERROR9 user=root")               # realistic log text, signature absent
    t4 = ("ab" * (n // 2), "ab" * (m // 2 - 1) + "ba")  # extra: periodic text
    names = ["T1 best case", "T2 worst case", "T3 realistic log (own choice)", "T4 periodic text
(extra)"]
    out = []
    for nm, (t, p) in zip(names, (t1, t2, t3, t4)):
        pos, c = brute_match(t, p)
        out.append({"test": nm, "n": len(t), "m": len(p), "alignments": len(t) - len(p) + 1,
                    "char_comparisons": c, "comparisons_per_alignment": round(c / (len(t) - len(p) + 1),
3),
                    "match_pos": pos, "predicted_formula": ""})
    out[0]["predicted_formula"] = "n-m+1 = %d" % (n - m + 1)
    out[1]["predicted_formula"] = "m(n-m+1) = %d" % (m * (n - m + 1))
    R["q3"] = out
    with open("raw_data_q3.csv", "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=out[0].keys()); w.writeheader(); w.writerows(out)

# ---------------------------------------------------------------- Q4
def closest_brute(P):
```

```
    n = len(P); best = float("inf"); pair = None; pairs = dcalc = upd = 0
    for i in range(n - 1):
        xi, yi = P[i]
        for j in range(i + 1, n):
            pairs += 1; dcalc += 1
            d = (xi - P[j][0]) ** 2 + (yi - P[j][1]) ** 2     # squared distance
            if d < best: best = d; pair = (i, j); upd += 1
    return best ** 0.5, pair, pairs, dcalc, upd

def q4():
    rows = []
    for n in (100, 500, 1000, 2000, 3000, 4000):
        P = [(random.uniform(0, 10000), random.uniform(0, 10000)) for _ in range(n)]
        t = time.perf_counter(); d, pr, pairs, dc, upd = closest_brute(P); t = time.perf_counter() - t
        rows.append({"n": n, "pairs_examined": pairs, "formula_n(n-1)/2": n * (n - 1) // 2,
                     "distance_calcs": dc, "best_updates": upd, "time_s": round(t, 4)})
    R["q4"] = rows
    with open("raw_data_q4.csv", "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=rows[0].keys()); w.writeheader(); w.writerows(rows)

# ---------------------------------------------------------------------- Q5
def hull_brute(P):
    """returns (ordered hull vertices, number of side tests). Points assumed distinct."""
    n = len(P); tests = 0; edges = []
    for i in range(n):
        for j in range(i + 1, n):
            (x1, y1), (x2, y2) = P[i], P[j]
            a, b, c = y2 - y1, x1 - x2, x1 * y2 - y1 * x2    # line a*x + b*y = c
            pos = neg = 0; ok = True
            for k in range(n):
                if k == i or k == j: continue
                tests += 1
                v = a * P[k][0] + b * P[k][1] - c
                if v > 0: pos += 1
                elif v < 0: neg += 1
                else:   # collinear: must lie inside segment Pi-Pj, otherwise Pi-Pj is not a full edge
                    if not (min(x1, x2) <= P[k][0] <= max(x1, x2) and min(y1, y2) <= P[k][1] <= max(y1,
y2)):
                        ok = False; break
                if pos and neg: ok = False; break
            if ok: edges.append((i, j))
    # build boundary: chain the edges into a cycle ordered counter-clockwise
    if not edges: return [], tests
    adj = {}
    for i, j in edges: adj.setdefault(i, []).append(j); adj.setdefault(j, []).append(i)
    verts = list(adj)
```

```
    if len(verts) == 2: return [P[v] for v in verts], tests          # all points collinear
    start = min(verts, key=lambda v: (P[v][0], P[v][1])); order = [start]; prev = None; cur = start
    while True:
        nxt = [v for v in adj[cur] if v != prev]
        if prev is None: nxt = [adj[cur][0]]
        n2 = nxt[0]
        if n2 == start: break
        order.append(n2); prev, cur = cur, n2
    return [P[v] for v in order], tests

def monotone_chain(P):  # ONLY used to cross-check correctness
    P = sorted(set(P))
    if len(P) <= 2: return P
    cr = lambda o, a, b: (a[0]-o[0])*(b[1]-o[1]) - (a[1]-o[1])*(b[0]-o[0])
    lo = []
    for p in P:
        while len(lo) >= 2 and cr(lo[-2], lo[-1], p) <= 0: lo.pop()
        lo.append(p)
    up = []
    for p in reversed(P):
        while len(up) >= 2 and cr(up[-2], up[-1], p) <= 0: up.pop()
        up.append(p)
    return lo[:-1] + up[:-1]

def q5():
    random.seed(11)
    setA = list({(random.randint(0, 100), random.randint(0, 100)) for _ in range(40)})
    # challenge set: square corners + points ON edges + duplicates removed + interior + near-
collinear
    setB = [(0,0),(10,0),(10,10),(0,10),(5,0),(10,5),(5,10),(0,5),(3,3),(7,7),(5,5),(2,8),(8,2),(1,1)]
    out = []
    for nm, P in (("A: random 40 pts", setA), ("B: square + collinear edge pts + interior", setB)):
        H, tests = hull_brute(P)
        ref = monotone_chain(P)
        # extreme vertices only (the checker removes collinear), compare as sets after dropping
collinear pts on edges
        def strip(H):
            if len(H) < 3: return set(H)
            keep = []
            for k in range(len(H)):
                a, b, c = H[k-1], H[k], H[(k+1) % len(H)]
                if (b[0]-a[0])*(c[1]-a[1]) - (b[1]-a[1])*(c[0]-a[0]) != 0: keep.append(b)
            return set(keep)
        out.append({"set": nm, "n": len(P), "hull_points_incl_collinear": len(H),
                    "extreme_vertices": len(strip(H)), "side_tests": tests,
                    "matches_reference": strip(H) == set(ref)})
```

```
        R.setdefault("q5_sets", {})[nm] = {"P": P, "H": H}
    R["q5"] = out
    # all-collinear and growth
    Hc, _ = hull_brute([(i, 2 * i) for i in range(6)])
    R["q5_collinear"] = Hc
    growth = []
    for n in (10, 20, 40, 80, 160):
        P = list({(random.randint(0, 10**6), random.randint(0, 10**6)) for _ in range(n)})
        t = time.perf_counter(); H, tests = hull_brute(P); t = time.perf_counter() - t
        growth.append({"n": len(P), "side_tests": tests, "n^3/2": len(P) ** 3 // 2, "hull_size": len(H),
"time_s": round(t, 4)})
    R["q5_growth"] = growth
    # convex-position (all points on hull) - parabola points, early-exit helps least here
    conv = []
    for n in (10, 20, 40, 80, 160):
        P = [(i, i * i) for i in range(n)]
        H, tests = hull_brute(P)
        conv.append({"n": n, "side_tests": tests, "n^3/2": n ** 3 // 2, "hull_size": len(H)})
    R["q5_convex"] = conv
    with open("raw_data_q5.csv", "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=growth[0].keys()); w.writeheader(); w.writerows(growth)
    # picture
    fig, axs = plt.subplots(1, 2, figsize=(9, 4.2))
    for ax, (nm, v) in zip(axs, R["q5_sets"].items()):
        P, H = v["P"], v["H"]
        ax.scatter([p[0] for p in P], [p[1] for p in P], s=18, c="tab:blue", label="all points")
        hx = [p[0] for p in H] + [H[0][0]]; hy = [p[1] for p in H] + [H[0][1]]
        ax.plot(hx, hy, "r-", lw=1.6, label="hull boundary"); ax.scatter(hx, hy, s=26, c="red",
zorder=3)
        ax.set_title(nm, fontsize=9); ax.legend(fontsize=7); ax.set_aspect("equal")
    plt.tight_layout(); plt.savefig("q5_hull.png", dpi=150); plt.close()

# ---------------------------------------------------------------- Q6
def knap_exhaustive(w, v, cap):
    n = len(w); best = -1; bestmask = 0; feas = 0
    for mask in range(1 << n):
        tw = tv = 0
        for i in range(n):
            if mask >> i & 1: tw += w[i]; tv += v[i]
        if tw <= cap:
            feas += 1
            if tv > best: best, bestmask = tv, mask
    return best, bestmask, feas, 1 << n

def q6():
    # verified small instance
```

```
    w = [5, 4, 6, 3]; v = [10, 40, 30, 50]; cap = 10
    allsubs = []
    for mask in range(16):
        ids = [i + 1 for i in range(4) if mask >> i & 1]
        allsubs.append((ids, sum(w[i-1] for i in ids), sum(v[i-1] for i in ids)))
    R["q6_small_table"] = allsubs
    best, mask, feas, tot = knap_exhaustive(w, v, cap)
    R["q6_small"] = {"best": best, "items": [i + 1 for i in range(4) if mask >> i & 1], "feasible":
feas, "total": tot}
    rows = []
    random.seed(5)
    for n in (10, 14, 16, 18, 20, 22):
        ww = [random.randint(5, 40) for _ in range(n)]; vv = [random.randint(10, 100) for _ in
range(n)]
        cap = sum(ww) // 2
        t = time.perf_counter(); b, m, f, tot = knap_exhaustive(ww, vv, cap); t = time.perf_counter() -
t
        rows.append({"n": n, "candidates_2^n": tot, "feasible": f, "best_value": b, "time_s": round(t,
3)})
    R["q6"] = rows
    with open("raw_data_q6.csv", "w", newline="") as f:
        wr = csv.DictWriter(f, fieldnames=rows[0].keys()); wr.writeheader(); wr.writerows(rows)

# ---------------------------------------------------------------- Q7  (redesign of Q4)
def closest_sweep(P):
    """Redesign: sort by x once; for each point only look right while dx^2 < best so far."""
    S = sorted(P); n = len(S); best = float("inf"); pair = None; dcalc = 0
    for i in range(n - 1):
        xi, yi = S[i]
        for j in range(i + 1, n):
            dx = S[j][0] - xi
            if dx * dx >= best: break
            dy = S[j][1] - yi; dcalc += 1
            d = dx * dx + dy * dy
            if d < best: best = d; pair = (S[i], S[j])
    return best ** 0.5, pair, dcalc

def q7():
    rows = []
    def gen(kind, n):
        if kind == "uniform": return [(random.uniform(0, 10000), random.uniform(0, 10000)) for _ in
range(n)]
        if kind == "clustered":
            cs = [(random.uniform(0, 10000), random.uniform(0, 10000)) for _ in range(8)]
            return [(random.gauss(c[0], 150), random.gauss(c[1], 150)) for c in (random.choice(cs) for
_ in range(n))]
```

```
    if kind == "same-x (adversarial)": return [(5000.0 + random.uniform(0, 1e-3),
random.uniform(0, 10000)) for _ in range(n)]
    if kind == "thin strip": return [(random.uniform(0, 10000), random.uniform(0, 50)) for _ in
range(n)]
  random.seed(99)
  for kind in ("uniform", "clustered", "thin strip", "same-x (adversarial)"):
    for n in (500, 1000, 2000, 4000):
      P = gen(kind, n)
      t = time.perf_counter(); d0, _, pairs, dc0, _ = closest_brute(P); t0 = time.perf_counter() - t
      t = time.perf_counter(); d1, _, dc1 = closest_sweep(P); t1 = time.perf_counter() - t
      assert abs(d0 - d1) < 1e-9
      rows.append({"distribution": kind, "n": n, "brute_dist_calcs": dc0, "sweep_dist_calcs":
dc1,
                   "reduction_x": round(dc0 / max(dc1, 1), 1), "brute_time_s": round(t0, 4),
"sweep_time_s": round(t1, 4)})
  R["q7"] = rows
  with open("raw_data_q7.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=rows[0].keys()); w.writeheader(); w.writerows(rows)

# ---------------------------------------------------------------- charts
def charts():
  q4 = R["q4"]
  plt.figure(figsize=(5, 3.4))
  plt.plot([r["n"] for r in q4], [r["pairs_examined"] for r in q4], "o-", label="measured pairs")
  ns = [r["n"] for r in q4]; plt.plot(ns, [n * (n - 1) / 2 for n in ns], "x--", label="n(n-1)/2")
  plt.xlabel("n drones"); plt.ylabel("pairs"); plt.legend(); plt.grid(alpha=.3); plt.tight_layout()
  plt.savefig("q4_pairs.png", dpi=150); plt.close()
  plt.figure(figsize=(5, 3.4))
  plt.plot(ns, [r["time_s"] for r in q4], "o-", color="tab:red"); plt.axhline(1.0, ls="--", c="gray",
label="1 s criterion")
  plt.xlabel("n drones"); plt.ylabel("time (s)"); plt.legend(); plt.grid(alpha=.3); plt.tight_layout()
  plt.savefig("q4_time.png", dpi=150); plt.close()
  q6 = R["q6"]
  plt.figure(figsize=(5, 3.4))
  plt.semilogy([r["n"] for r in q6], [r["candidates_2^n"] for r in q6], "o-", label="candidates 2^n")
  plt.semilogy([r["n"] for r in q6], [max(r["time_s"], 1e-3) for r in q6], "s-", label="time (s)")
  plt.xlabel("n packages"); plt.legend(); plt.grid(alpha=.3, which="both"); plt.tight_layout()
  plt.savefig("q6_growth.png", dpi=150); plt.close()
  q7 = [r for r in R["q7"] if r["distribution"] == "uniform"]
  plt.figure(figsize=(5, 3.4))
  plt.plot([r["n"] for r in q7], [r["brute_dist_calcs"] for r in q7], "o-", label="brute force")
  plt.plot([r["n"] for r in q7], [r["sweep_dist_calcs"] for r in q7], "s-", label="redesign (sort +
prune)")
  plt.yscale("log"); plt.xlabel("n"); plt.ylabel("distance calculations"); plt.legend();
plt.grid(alpha=.3, which="both"); plt.tight_layout()
  plt.savefig("q7_compare.png", dpi=150); plt.close()
```

```
    ds = list(dict.fromkeys(r["dataset"] for r in R["q1"]))
    fig, ax = plt.subplots(figsize=(6.4, 3.4)); wd = 0.27
    for k, alg in enumerate(("Selection", "Bubble", "Insertion")):
        vals = [next(r["comparisons"] for r in R["q1"] if r["dataset"] == d and r["algorithm"] == alg)
for d in ds]
        ax.bar([i + (k - 1) * wd for i in range(len(ds))], vals, wd, label=alg)
    ax.set_yscale("log"); ax.set_xticks(range(len(ds))); ax.set_xticklabels([d.replace(" ", "\n", 1) for
d in ds], fontsize=6)
    ax.set_ylabel("comparisons (log)"); ax.legend(fontsize=7); plt.tight_layout();
plt.savefig("q1_comparisons.png", dpi=150); plt.close()
    q3 = R["q3"]
    plt.figure(figsize=(5, 3.2))
    plt.bar([r["test"].split(" ")[0] for r in q3], [r["char_comparisons"] for r in q3],
color=["tab:green", "tab:red", "tab:blue", "tab:orange"])
    plt.yscale("log"); plt.ylabel("character comparisons (log)"); plt.tight_layout();
plt.savefig("q3_comparisons.png", dpi=150); plt.close()

if __name__ == "__main__":
    q1(); q2(); q3(); q4(); q5(); q6(); q7(); charts()
    json.dump(R, open("results.json", "w"), indent=1, default=str)
    print("done")
```

Appendix B: Raw Experimental Data

raw_data_q1.csv

dataset	algorithm	n	comparisons	swaps_or_shifts	time_ms
Random	Selection	1000	499500	995	20.224
Random	Bubble	1000	499149	248445	38.651
Random	Insertion	1000	249438	248445	17.992
Reverse (worst)	Selection	1000	499500	500	19.939
Reverse (worst)	Bubble	1000	499500	499500	49.641
Reverse (worst)	Insertion	1000	499500	499500	35.386
Already sorted	Selection	1000	499500	0	18.62
Already sorted	Bubble	1000	999	0	0.046
Already sorted	Insertion	1000	999	0	0.072
95% sorted + 5% new	Selection	1000	499500	962	18.448
95% sorted + 5% new	Bubble	1000	496799	23326	22.489
95% sorted + 5% new	Insertion	1000	24325	23326	2.088

All equal (edge)	Selection	1000	499500	0	17.895
All equal (edge)	Bubble	1000	999	0	0.043
All equal (edge)	Insertion	1000	999	0	0.066

raw_data_q2.csv

n	case	comparisons
100	first	1
100	middle	51
100	last	100
100	unsuccessful	100
100	avg of 1000 random successful	51.348
1000	first	1
1000	middle	501
1000	last	1000
1000	unsuccessful	1000
1000	avg of 1000 random successful	504.684
10000	first	1
10000	middle	5001
10000	last	10000
10000	unsuccessful	10000
10000	avg of 1000 random successful	4937.89
100000	first	1
100000	middle	50001
100000	last	100000
100000	unsuccessful	100000
100000	avg of 1000 random successful	51052.593

raw_data_q3.csv

test	n	m	alignments	char_comparisons	comparisons_per_alignment	match_pos	predicted_formula
T1 best case	10000	20	9981	9981	1.0	-1	n-m+1 = 9981
T2 worst case	10000	20	9981	199620	20.0	-1	m(n-m+1) = 199620
T3 realistic log (own choice)	10000	16	9985	10839	1.086	-1	
T4 periodic text	10000	20	9981	99819	10.001	-1	

(extra)							

raw_data_q4.csv

n	pairs_examined	formula_n(n-1)/2	distance_calcs	best_updates	time_s
100	4950	4950	4950	7	0.0007
500	124750	124750	124750	14	0.019
1000	499500	499500	499500	16	0.1129
2000	1999000	1999000	1999000	16	0.3699
3000	4498500	4498500	4498500	10	0.6696
4000	7998000	7998000	7998000	20	1.2591

raw_data_q5.csv

n	side_tests	n^3/2	hull_size	time_s
10	167	500	7	0.0001
20	894	4000	10	0.0002
40	4817	32000	11	0.0009
80	19097	256000	10	0.0035
160	96850	2048000	14	0.0166

raw_data_q6.csv

n	candidates_2^n	feasible	best_value	time_s
10	1024	512	372	0.001
14	16384	8270	540	0.011
16	65536	32768	586	0.049
18	262144	131072	733	0.226
20	1048576	528183	787	1.062
22	4194304	2109451	1034	4.479

raw_data_q7.csv

distribution	n	brute_dist_calcs	sweep_dist_calcs	reduction_x	brute_time_s	sweep_time_s
uniform	500	124750	333	374.6	0.0238	0.0003
uniform	1000	499500	725	689.0	0.0794	0.0005
uniform	2000	1999000	2742	729.0	0.392	0.0012
uniform	4000	7998000	7598	1052.6	1.0055	0.0023
clustered	500	124750	519	240.4	0.0202	0.0002
clustered	1000	499500	510	979.4	0.0762	0.0004
clustered	2000	1999000	2885	692.9	0.3089	0.001
clustered	400	7998000	4495	1779.3	1.2236	0.0019

	0					
thin strip	500	124750	52	2399.0	0.019	0.0002
thin strip	1000	499500	100	4995.0	0.0739	0.0003
thin strip	2000	1999000	112	17848.2	0.291	0.0007
thin strip	4000	7998000	318	25150.9	1.2474	0.0014
same-x (adversarial)	500	124750	124750	1.0	0.0197	0.016
same-x (adversarial)	1000	499500	499500	1.0	0.1144	0.0545
same-x (adversarial)	2000	1999000	1999000	1.0	0.4004	0.2199
same-x (adversarial)	4000	7998000	7998000	1.0	1.2469	0.9832